MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
Akademik Dönem 2022-2023, Veri Yapıları
BÜTÜNLEME TELAFİ ÖDEVİ

DERSİ VEREN ÖĞRETİM ELEMANI:	İMZA:	**Veriliş Tarihi: 27/02/2023 Saat: 11.00**
ÖĞRENCİ ADI SOYADI: **ÖĞRENCİ NO :**	İMZA	**Teslim Tarihi: 28/02/2023 Saat: 17.00**

Soru	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**TOPLAM PUAN:**
Puan											
Program Çıktısı	**PÇ1, PÇ2**	**PÇ1, PÇ2**	**PÇ5**	**PÇ7**							

1)

```
struct bt_dugum{ // ikili arama ağacı
   int anahtar;
   struct bt_dugum *sol,*sag;
};

struct ls_dugum{ // bağlantılı liste
   struct bt_dugum *dugum;
   struct ls_dugum *sonraki;
 };
```

İkili arama ağacı ve bağlantılı listeye ait düğüm tanımlamaları yanda verilmiştir. Buna göre, ikili arama ağacının kökünden anahtarı verilen düğüme kadar olan yol üzerinde bulunan düğümleri listeye ekleyen ve tanımlaması aşağıda verilen fonksiyonu yazınız? **(25 puan)**

void liste_olustur(struct bt_dugum *kok, int anahtar, struct ls_dugum **liste_basi){

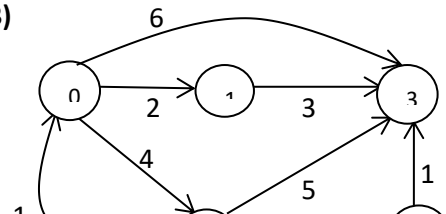
2)

```
struct cell {
   char *anahtar;
   struct cell *next;
};
struct table_node{
   int count;
   struct cell *header;
};
struct hash_tablosu{
   struct table_node *tablo_basi;
   int tablo_uzunlugu;
   int multiplier;
};
unsigned int hash(char *s, int multiplier, int table_size){
   int i=0;
   unsigned int value =0;
   while(s[i] != '\0'){
      value=(s[i]+multiplier*value) % table_size;
i++;
   }
   return value;
}
```

Hash tablosu ile ilgili veri tanımlaması solda verilmektedir. Buna göre parametre olarak anahtarı verilen düğümün bulunduğu listedeki anahtarı en küçük olan düğümü silen ve tanımlaması aşağıda verilen fonksiyonu yazınız? **(25 puan)**

void liste_sil(struct hash_tablosu *htable, char *anahtar){

3)



2

MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
Akademik Dönem 2022-2023, Veri Yapıları
BÜTÜNLEME TELAFİ ÖDEVİ

DERSİ VEREN ÖĞRETİM ELEMANI:	İMZA:	**Veriliş Tarihi: 27/02/2023 Saat: 11.00**
ÖĞRENCİ ADI SOYADI: **ÖĞRENCİ NO :**	İMZA	**Teslim Tarihi: 28/02/2023 Saat: 17.00**

Soru	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**TOPLAM PUAN:**
Puan											
Program Çıktısı	**PÇ1, PÇ2**	**PÇ1, PÇ2**	**PÇ5**	**PÇ7**							

Yukarıda bir yönlü graf ve bu grafa ait kenar ağırlıkları verilmektedir. Buna göre,
a) Bu grafa ait bitişiklik matrisini yazınız? **(5 puan)**

b)

```
for(k = 0; k < n; k++) {
    if(k==2) break;
     for(i = 0; i < n; i++) {
        if(d[i][k] == MAXINT) continue;
        for(j = 0; j < n; j++) {
           if(d[k][j] == MAXINT) continue;
           newdist = d[i][k] + d[k][j];
           if(newdist < d[i][j])
              d[i][j] = newdist;
        }
    }
```

Yandaki kodda d dizisinin yukarıdaki grafa ait bitişiklik matrisi olduğunu kabul edelim (n düğüm sayısı 6). Buna göre yandaki kod çalıştırıldıktan sonra d dizisini (matrisini) yazınız? **(10 puan)**

c) b) şıkkındaki kod parçacığındaki **if(k==2) break;** satırını silersek, bu durumda hesaplanan d dizisini (matrisini) yazınız? **(10 puan)**

4)Aşağıdaki soruları cevaplayınız.

a) n düğüm sayısı olmak üzere AVL ağacına düğüm ekleme algoritmasının karmaşasını O notasyonunda ifade ediniz? **(5 puan)**

b) n düğüm sayısı olmak üzere AVL ağacının yüksekliğini O notasyonunda ifade ediniz? **(5 puan)**

c)MinHeap veri yapısında eleman ekleme işleminin karmaşıklığını O notasyonunda ifade ediniz?**(5 puan)**

d) "Minimum Spanninnig Tree (En küçük kapsama ağacı) yönsüz graflar için hesaplanır", ifadesinin doğru olup olmadığını **Evet** veya **Hayır** şeklinde cevaplayınız? **(5 puan)**

e)Minimum spanninnig tree ve shortest path algoritmalarına birer örnek veriniz? (Sadece bu algoritmaların isimlerini yazınız?) **(5 puan)**

1.

```
void liste_olustur(struct bt_dugum *kok, int anahtar, struct ls_dugum **liste_basi){

    struct bt_dugum * kok_copy = kok;
    while(kok_copy != NULL){
        if(kok_copy->anahtar > anahtar)
            kok_copy = kok_copy->sag;
        else if(kok_copy->anahtar < anahtar)
            kok_copy = kok_copy->sol;
        else{
            break;
        }
    }

    if(kok_copy == NULL){
        free(*liste_basi);
        *liste_basi = NULL;
        return;
    }

    *liste_basi = (struct ls_dugum *)malloc(sizeof(struct ls_dugum));
    (*liste_basi)->dugum = kok;
    (*liste_basi)->sonraki = NULL;
    struct ls_dugum * copy = *liste_basi;
    while(kok != NULL){
        if(kok->anahtar > anahtar)
            kok = kok->sag;
        else if(kok->anahtar < anahtar)
            kok = kok->sol;
        else{
            break;
        }
        struct ls_dugum * temp = (struct ls_dugum *)malloc(sizeof(struct ls_dugum));
        temp->dugum = kok;
        temp->sonraki = NULL;
        copy->sonraki = temp;
        copy = temp;
    }
}
```

2.

```
void liste_sil(struct hash_tablosu *htable, char *anahtar){
    if(htable == NULL || htable->tablo_uzunlugu == 0 || anahtar == NULL)
        return;

    unsigned int hashvalue = hash(anahtar, htable->multiplier, htable->tablo_uzunlugu);

    int bulunamadi = 1;
    struct Cell * iterate = htable->tablo_basi[hashvalue].tablo_basier;
    for(int i = 0; i < htable->tablo_basi[hashvalue].count; i++){
        if(strcmp(iterate->anahtar, anahtar) == 0)
            bulunamadi = 0;
    }
    if(bulunamadi)
        return;

    struct Cell * prev = NULL;
    struct Cell * enkucukOnce = NULL;
    struct Cell * enkucuk = htable->tablo_basi[hashvalue].tablo_basier;
    iterate = htable->tablo_basi[hashvalue].tablo_basier->next;

    while(iterate != NULL){
        if(strcmp(iterate->anahtar, enkucuk->anahtar) < 0){
            enkucukOnce = prev;
            enkucuk = iterate;
        }
        prev = iterate;
        iterate = iterate->next;
    }

    if(enkucukOnce == NULL){
        htable->tablo_basi[hashvalue].tablo_basier = htable->tablo_basi[hashvalue].tablo_basier->next;
    }else{
        enkucukOnce->next = enkucuk->next;
    }

    free(enkucuk->anahtar);
    free(enkucuk);
    htable->tablo_basi[hashvalue].count--;
}
```

3.a)
$$\begin{bmatrix} 0 & 2 & 4 & 6 & \infty & \infty \\ \infty & 0 & \infty & 3 & \infty & \infty \\ 1 & \infty & 0 & 5 & 2 & \infty \\ \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & \infty & 1 & \infty & 0 \end{bmatrix}$$

3.b)
$$\begin{bmatrix} 0 & 2 & 4 & 5 & \infty & \infty \\ \infty & 0 & \infty & 3 & \infty & \infty \\ 1 & 3 & 0 & 5 & 2 & \infty \\ \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & \infty & 1 & \infty & 0 \end{bmatrix}$$

3.c)
$$\begin{bmatrix} 0 & 2 & 4 & 5 & 6 & 9 \\ \infty & 0 & \infty & 3 & \infty & \infty \\ 1 & 3 & 0 & 5 & 2 & 5 \\ \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & 4 & 0 & 3 \\ \infty & \infty & \infty & 1 & \infty & 0 \end{bmatrix}$$

4.a) **O(log n)**

b) **O(log n)**

c) **O(log n)**

d)Evet

5.Shortest Path: Bellman Ford, Floyd Warshall, Dijkstra.

Minimum spanninnig tree: Prim, Kruskal